

Assignment Report

Full Name : Jack Millington	Student ID: s5405915
Course Name: 2801 – Computing Algorithms	Date: 25.05.25

I. Overview

- Programming Languages: Python.
- This assignment consists of four real world problems that are related to data structures, sorting algorithms, searching systems, and graph algorithms.

II. Problem 1

A restaurant manages incoming online food delivery orders. The system needs to handle orders in a first-come, first-served manner while allowing customers to cancel orders before they are processed. Suppose that there are no priority levels, no need for reordering, and no fixed number of order slots.

Task 1 – Implement the queue using an array:

The queue should support:

- Add order: Enqueue a new order.
- Process order: Dequeue and remove the order from the queue.
- View next order: Peek at the next order without removing it.
- Cancel order: Remove a specific order from the queue.
- Check if empty: Determine if there are pending orders.

Answer:

```
# Task 1
class ArrayQueue:
    # Initialises queue as an array
    def __init__(self):
        self.queue = []
    # adds order to end of queue
    def addOrder(self, order):
        self.queue.append(order)
    # pops the element at the front of the queue
    def processOrder(self):
        if self.isEmpty():
            return None
        return self.queue.pop(0)
    # views the element at the front of the queue
    def viewNextOrder(self):
        return self.queue[0]
    # removes a specified order from the queue,
    # moves everything behind it forwards by 1 in the queue
    def cancelOrder(self, order):
        if order in self.queue:
            self.queue.remove(order)
            return True
        return False
    # checks if the queue is empty
    def isEmpty(self):
        return len(self.queue) == 0
```

Figure 1: Array Queue Implementation

Task 2 - Discuss the limitations of using an array-based queue for this implementation:

Answer:

An array-based queue has some limitations:

To remove an order from the middle of the queue, it requires $O(n)$ time complexity, as it must loop through the array until it finds the correct element. It then must shift all elements after it. When an order is dequeued from the front, all elements still need to be shifted forwards, requiring $O(n)$ time complexity, instead of $O(1)$. As python arrays dynamically change size, sometimes when adding an element to the back of the queue, the current array size reaches capacity, so python creates a new array double the size and copies all elements across which requires $O(n)$ time. Otherwise, adding an element to the back of the queue should usually be $O(1)$ time.

Task 3 - Improvements:

Q1: Given the identified limitations of an array-based queue, would a singly linked list be a better alternative? Why or why not?

Answer:

In a singly linked list, dequeuing an element from the front can be done in $O(1)$ time, rather than $O(n)$. Enqueueing an element to the back will always be in constant time ($O(1)$). A linked list dynamically allocates memory unlike an array that needs to resize which prevents unnecessary memory usage.

However, there are still some drawbacks.

Cancelling an order from the middle of the queue is still $O(n)$ as we still need to search through the queue to find the element. Each node in a linked list also requires memory for storage of a pointer to the next node, thus is less space efficient than an array. Overall, a singly linked list is better than an array-based queue if there are frequent enqueue and dequeue operations mainly due to not needing to shift all elements. However, order cancellation time is still not improved, and it also uses more memory due to the need to store pointers for each node.

Q2: Implement the queue using a singly linked list and analyse its benefits and drawbacks.

Answer:

Implementing a queue using A singly linked list has some benefits that can be addressed. Enqueuing an element to the back and dequeuing an element from the front can be done in constant time ($O(1)$). It also dynamically allocates memory, meaning there is no need for resizing or wasted memory which may happen when using an array.

Drawbacks of a singly linked list are still apparent, however. Removing an order from the middle of the queue still runs in $O(n)$ time, due to the need to traverse the list to find the element. It also requires extra memory usage as each node stores a pointer to the next, and is therefore less space efficient than an array. Overall, as stated previously, a singly linked list is better than an array based queue if enqueue and dequeue operations are frequent, though order cancellation time is not improved and memory usage is increased.

Below is my implementation of the singly linked list.

```
class SinglyLinkedListQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    # adds an order to the back of the queue
    def addOrder(self, order):
        new_node = Node(order)
        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

    # returns order at front of queue and
    # sets the front to the next node in queue
    def processOrder(self):
        if self.isEmpty():
            return None
        temp = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        return temp.order

    # returns the order at the front
    def viewNextOrder(self):
        if self.isEmpty():
            return None
        return self.front.order

    # linear search to find order,
    # then removes it from queue,
    # and sets pointer of node in front to node
    # behind the one that just got cancelled
    def cancelOrder(self, order):
        if self.isEmpty():
            return False
        temp = self.front
        prev = None
        while temp is not None:
            if temp.order == order:
                if prev is None:
                    self.front = temp.next
                else:
                    prev.next = temp.next
                if temp == self.rear:
                    self.rear = prev
                return True
            prev = temp
            temp = temp.next
        return False

    # Checks if queue is empty
    def isEmpty(self):
        return self.front is None
```

Figure 2: Singly linked list queue implementation

Task 4 – Optimised Implementation:

Suppose that each order in the system has a direct reference, meaning we can instantly locate any specific order without searching through the queue. Would a doubly linked list improve upon a singly linked list in this case? Why or why not?

Answer:

Yes, a doubly linked list would improve performance for order cancellation. This is because it changes this operation to constant time $O(1)$, rather than $O(n)$ like it is in an array or singly linked list. The reason for this is that each order now has a direct reference and therefore we do not have to traverse the array to find it. However, a doubly linked list will require double the memory usage as each node stores a pointer to its previous element, as well as its next element.

Analyse and compare all three implementations (array-based queue, singly linked list, and doubly linked list) based on:

- Time complexity for cancelling an order.
- Space complexity and memory usage.
- Situations where each implementation is preferred

Time complexity for cancelling an order:

Array: $O(n)$ as it has to search for the element and then shift all elements after it.

Singly Linked List: $O(n)$ as it requires traversal through the list.

Doubly Linked List: $O(1)$ as there is a direct reference to the order so no traversal is required.

Space complexity and memory usage:

Array: $O(n)$, This is the most memory efficient as there are no extra pointers stored, the only drawback regarding space complexity is the dynamically changing array sizes which can be sometimes over allocated, leaving unused memory.

Singly Linked List: $O(n)$, This is the second most efficient as it requires storage of a pointer for each node. However, there is no over allocation of memory, which an array can do.

Doubly Linked List: $O(n)$, This is the least memory efficient as it requires storage of two pointers for each node, using the most memory per order, however, there is no over allocation of memory, which an array can do.

Situations where each implementation is preferred:

An array based queue implementation is best used in a situation where memory is very limited and order cancellations are rare.

A singly linked list is best used when orders are frequently being added and dequeued.

A doubly linked list is best used when orders within the middle of the queue need to be cancelled frequently and each order has a direct reference.

Benchmarking:

The array queue, singly linked list, and doubly linked list were benchmarked, to further support the theory. This was done for 10000 elements.

Operation	Array	SinglyLL	DoublyLL
processOrder	0.006562	0.000959	0.000981
viewNextOrder	0.000284	0.000570	0.000527
cancelOrder	0.005018	0.001012	0.000578
isEmpty	0.000392	0.000237	0.000266

Figure 3: Queue benchmarking time complexity

As can be seen, the singly and doubly linked list performed much faster than the array queue when processing an order. As this operation becomes $O(1)$ rather than $O(n)$. The cancel order operation runs fastest in a doubly linked list as it runs in $O(1)$ time, rather than $O(n)$ like the array queue and singly linked list. The singly linked list still performs much faster than the array in this as there are much more steps when shifting all elements in an array, compared to reassigning exactly one or 2 pointers. All other operations run relatively the same speed, as they are all $O(1)$. Interestingly, the array was fastest when accessing the next order, this may be because it is simply a C-level list lookup, which is an instant action, whereas the linked list requires dereferencing of a python object pointer.

###-----END PROBLEM 1-----###

III. Problem 2

This section focuses on implementing and analysing various sorting algorithms to determine their efficiency across different scenarios. You will benchmark their performance on different input sizes and types of lists, visualize the results, and explore hybrid sorting techniques

Task 1:

Benchmark Sorting Algorithms:

- Implement the following sorting algorithms:
 - QuickSort
 - MergeSort
 - HeapSort
 - InsertionSort
 - SelectionSort
- Test their performance on lists of various sizes:
 - 1000, 5000, 10000, 20000, 30000, 40000, 50000, 100000, and 500000 elements.
- Run tests on different types of input data:
 - Random lists
 - Reverse sorted lists
 - Lists with extreme value distributions (e.g., mostly small values with a few large outliers)
 - Lists with many duplicate values
- Visualize execution time results using line graphs.

Figure 4: Problem 2 Task 1 Specification

Answer:

The sorting algorithms, quick sort, merge sort, heap sort, insertion sort and selection sort were implemented and benchmarked. Originally, the pivot point for the quick sort algorithm was chosen as the end element of the list, but when benchmarked on the reverse sorted lists, it would hit python's recursion depth limit and result in a recursion error. This is because it results in partitions of size $n - 1$ and 0 at every step, leading the recursion depth to become $O(n)$ growing linearly, instead of $O(\log n)$, growing logarithmically, which quickly leads to a recursion error. The fix for this was to implement a randomly chosen pivot point, this led to increased efficiency, and the code to run without errors.

Here is the algorithm implementation:

```
# Task 1:
import random
import time
import matplotlib.pyplot as plt

# Quick Sort:
def quickSort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[random.randint(0, len(arr) - 1)]
```

```

l = []
e = []
g = []
for num in arr:
    if num > pivot:
        g.append(num)
    elif num < pivot:
        l.append(num)
    else:
        e.append(num)

leftSorted = quickSort(l)
rightSorted = quickSort(g)
return leftSorted + e + rightSorted

# MergeSort
def mergesort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = mergesort(arr[:mid])
    right = mergesort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Heapsort
def heapsort(arr):
    def heapify(arr, n, i):
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and arr[i] < arr[left]:
            largest = left
        if right < n and arr[largest] < arr[right]:
            largest = right
        if largest != i:
            arr[i], arr[largest] = arr[largest], arr[i]
            heapify(arr, n, largest)

    n = len(arr)
    for i in range(n // 2, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
    return arr

# Insertion Sort
def insertionsort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

# Selection Sort
def selectionsort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

```

Below are the benchmarks for random lists, reverse sorted lists, lists with outliers, and lists with multiple duplicate values. As insertion and selection sort run in $O(n^2)$ time. They would take an unnecessary amount of time on large element lists. So the highest list length tested for these sorting algorithms was 20000 elements, as this already took about 15 seconds to run in python each. Both the x and y axis are scaled logarithmically so it is

visually easier to interpolate the data from the graph. It is also worth mentioning that these results are based on a python implementation and may differ to results in another language. This is because some python methods, such as `append()`, and list slicing, are optimised C extensions, meaning the python method is running in C behind the scenes, which runs much faster because it is a lower level language, and could skew the data.

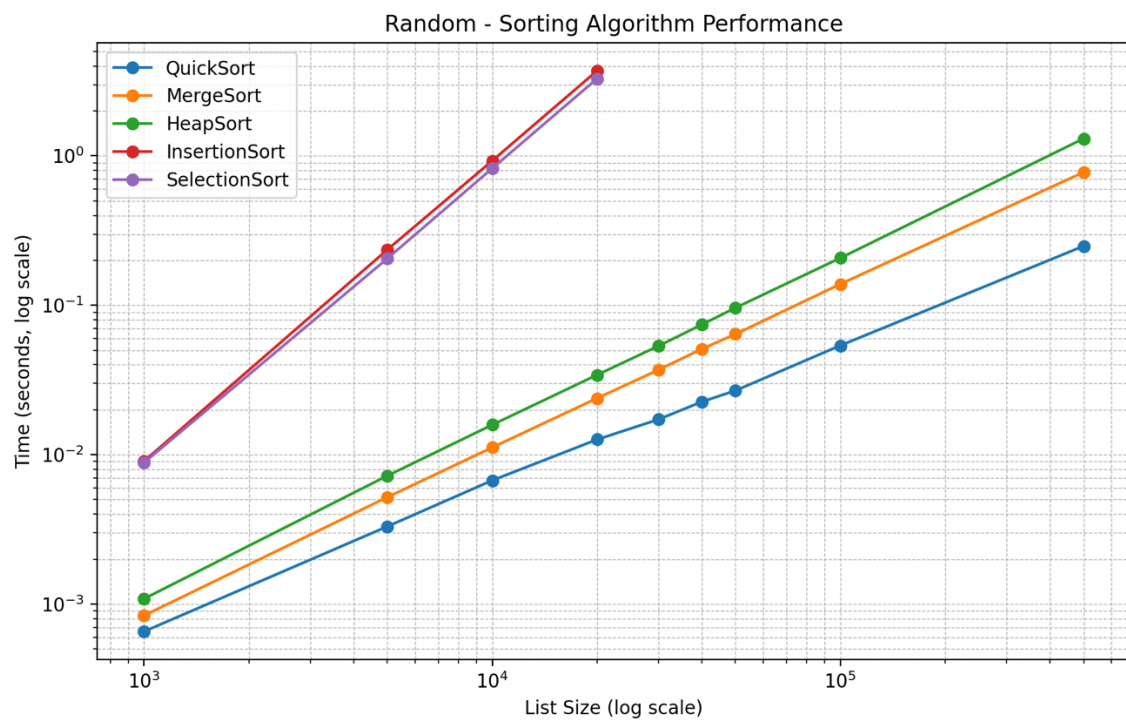


Figure 5: Benchmark on random lists

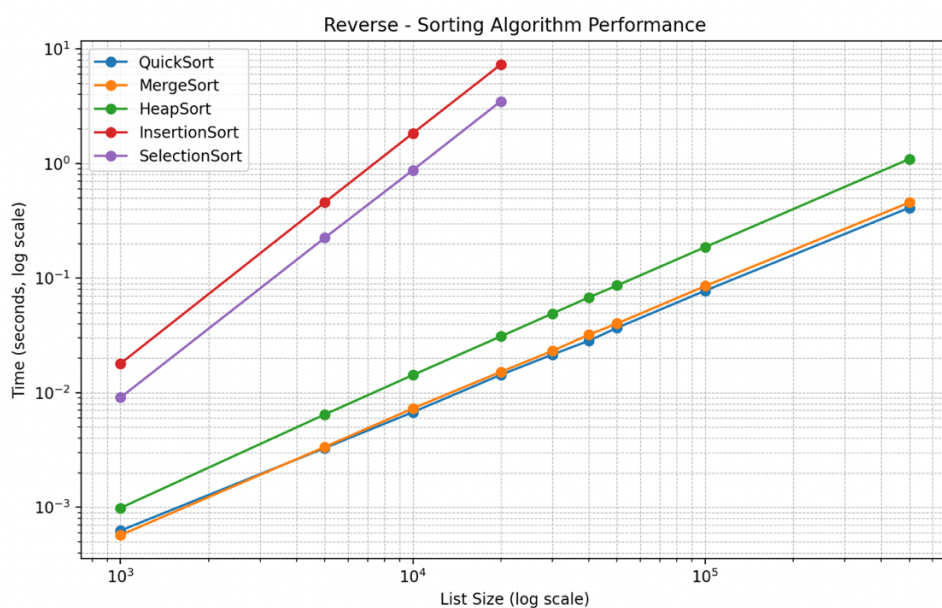


Figure 6: Benchmark on reverse sorted lists

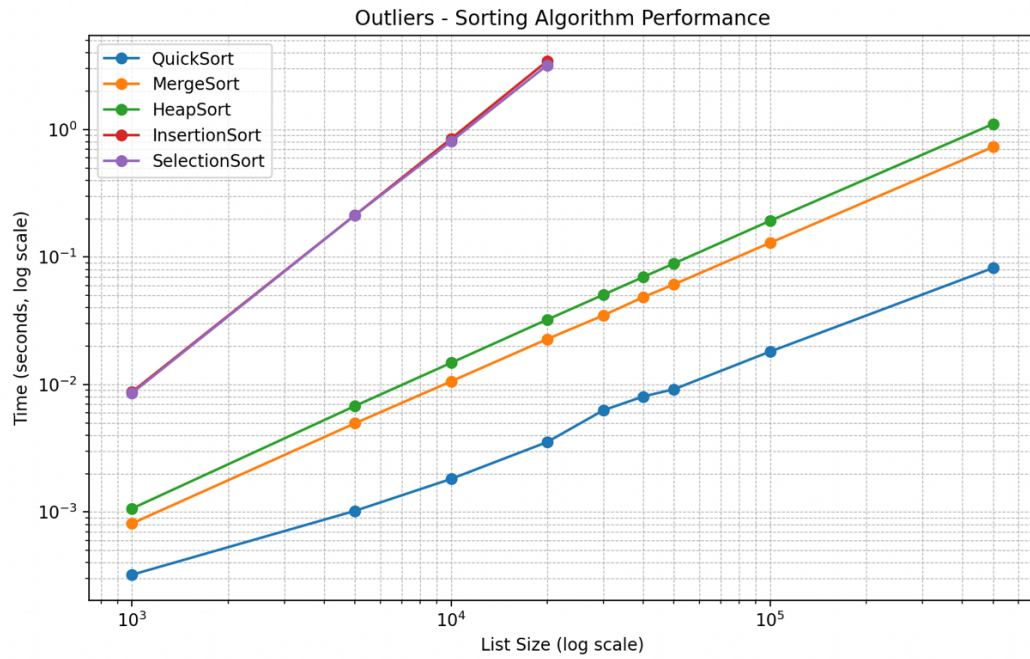


Figure 7: Benchmark on lists with extreme outliers

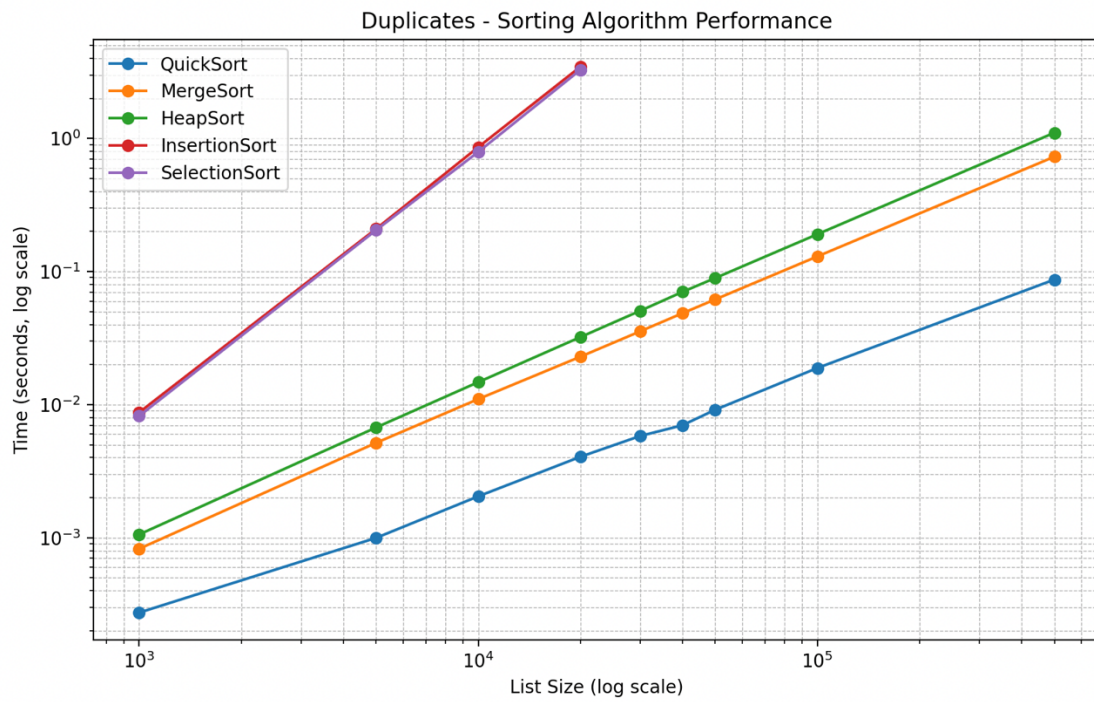


Figure 8: Benchmark on lists with many duplicates

Task 2: Scenario Based analysis.

Q1. Based on the benchmarks, identify which sorting algorithm performs best under different conditions.

It seems like quicksort consistently performs the best under all conditions tested in this benchmark, though was slightly closer with merge sort when testing on reverse sorted lists. Though, mergeSort is more stable as it maintains the order of equal elements, so mergeSort may be preferable under certain circumstances where stable sorting is required. An example of this would be secondary sorting, such as ordering by last name then first name, as items with equal keys could get shuffled randomly, breaking the secondary ordering.

Q2. Provide an explanation of why certain algorithms perform better in some cases.

On the outliers and duplicates, quicksort performed extremely well compared to its other $O(n \log n)$ counterparts. This could be because random pivot selection often splits the arrays into relatively balanced sub-arrays, and the three-way-partitioning approach on my algorithm helps to group duplicates quickly and reduces extra recursion. The quick, merge, and heap sort are all algorithms with a $O(n \log n)$ time complexity and thus run much faster than insertion and selection sort which run in $O(n^2)$ time complexity.

Selection sort does perform well in certain use cases. One case is for finding the lowest k elements in an array. This is because selection sort always finds the minimum in each pass in $O(n)$ time. This means that finding the lowest k elements will take $O(kn)$ time, rather than needing to sort the entire array like the other algorithms. This can be faster than $O(n \log(n))$ algorithms, depending on how large of a number k is.

Even though merge and heap sort guarantee $O(n \log n)$ time complexity, while quicksort is $O(n \log n)$ at its best case and $O(n^2)$ at its worst case. Quicksort still remains faster in practice because heapsort has worse cache efficiency, more swaps and more overhead per operation, and merge sort requires more data movement, and compares elements in already sorted sub arrays.

Q3. In what scenario does quicksort perform poorly. Analyse cases where Quicksort has high time complexity and explain why it happens.

Quicksort performed most poorly on reverse sorted lists but still was slightly faster than all other sorting algorithms including merge sort. This may be simply due to suboptimal pivot selection; it was still the fastest algorithm in practice. In the absolute worst-case scenario where the pivot that is randomly selected is a minimum or maximum on every single iteration, this would near $O(n^2)$ time complexity. This happens as each sub array partition is completely imbalanced, resulting in a plethora of recursions. This is highly unlikely when using random pivot selection and the chances of this are near zero.

However, if the pivot selection was based on the last element of the array, quicksort would perform extremely poorly. As one of L and G has size $n - 1$ and the other has size 0. This can likely result in a stack overflow and a crash of the program.

Task 3: Improving QuickSort

Q1. Can we improve quicksort's performance where it performs poorly? Design and implement an optimised version of QuickSort for each case where it underperforms.

Some limitations of my current method include:

The possibility of poor pivot selection as it's randomly chosen and can lead to $O(n^2)$ time complexity. A solution for this is to use a median of three pivot selection which would decrease the possibility of picking a bad pivot. This is because it reduces the possibility of it choosing a pivot that is the worst-case scenario, as it chooses the median element out of the three randomly chosen ones.

Recursive calls on very small subarrays can add unnecessary overhead. This can be limited by switching to insertion sort for subarrays less than 10. My optimised version below implements a hybrid quicksort version which uses a randomly selected median of 3 pivot and insertion sort for array sizes 10 or below, and still using three-way partitioning..

```
# Insertion Sort for small subarrays
def insertion_sort(arr, left, right):
    for i in range(left + 1, right + 1):
        key = arr[i]
        j = i - 1
        while j >= left and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

def median_of_three(arr, left, right):
    indices = random.sample(range(left, right + 1), 3)
    values = [arr[i] for i in indices]
    values.sort()
    return values[1]

def quick_sort_helper(arr, left, right):
    if right - left < 10:
        insertion_sort(arr, left, right)
        return

    pivot = median_of_three(arr, left, right)
    l, e, g = [], [], []
    for num in arr[left:right + 1]:
        if num > pivot:
            g.append(num)
        elif num < pivot:
            l.append(num)
        else:
            e.append(num)
    arr[left:right + 1] = quickSortn(l) + e + quickSortn(g)

def quickSortn(arr):
    quick_sort_helper(arr, 0, len(arr) - 1)
    return arr
```

Figure 9: Improved Quick Sort

Q2. Test the optimised version and compare it to the standard QuickSort.

The following has been benchmarked. Where quickSortn is the new and improved model, and quicksort is the model used in the previous benchmarks. The models performed almost exactly the same, except for the benchmark on arrays with many duplicates, where it proved to be much more stable.

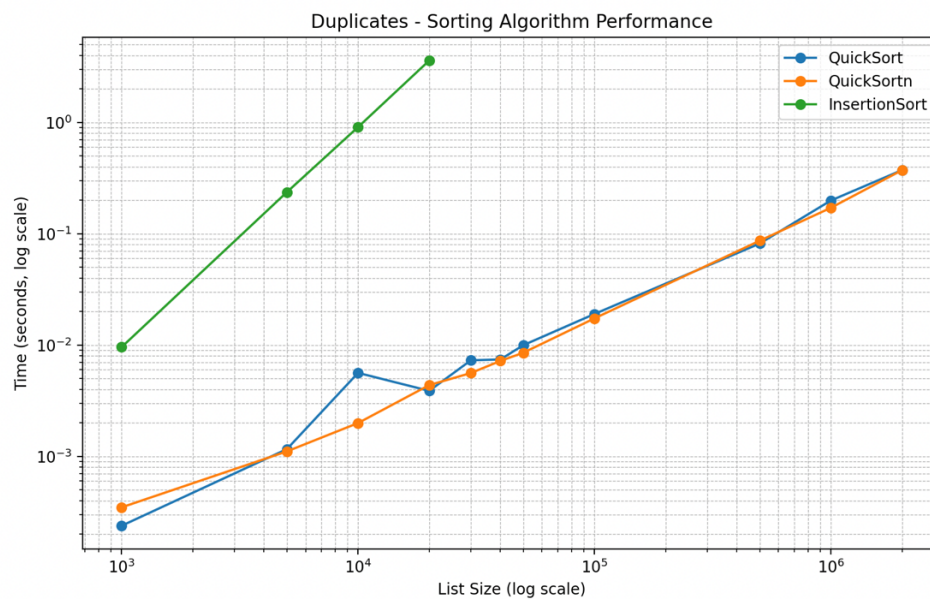


Figure 10: Improved quick sort vs original on random lists with duplicates

Although the improved version does not seem faster, it is about the same speed and is much more consistent. As can be seen the original version had major changes in time during this duplicates benchmark, while the new and improved version stayed very consistent.

The reason for this improvement is that picking the pivot as the median of three randomly sampled elements drastically reduces the chances of hitting a partition of $n-1$ and 0. It tends to get splits closer to balanced on average, letting each recursive step do roughly the same amount of work. The hybrid of switching to insertion sort when there are less than 10 elements left limits the overhead of recursive partitioning on very small sub-arrays, since insertion sort on small chunks is more predictable and cache friendly than recursive quick sort calls.

IV. Problem 3

Due to an accident in a warehouse, a large collection of items lost their labels and now must be identified solely by their weight. The warehouse system needs to quickly find an object with a specific weight to fulfill customer orders. However, searching for an item manually through the entire collection is inefficient, so an organized system is needed to allow fast searches while also handling items being relocated or removed from the inventory over time.

Task 1: Implement a searching system:

Q1: Choose a suitable data structure to organise items efficiently.

The data structure I implemented for the warehouse is a sorted array. I implemented this by actively inserting each new element into a sorted position in the array. This is an appropriate data structure of choice as it allows for fast searches and enables the use of binary search algorithms.

```
def add_item(self, weight):
    i = 0
    while i < len(self.items) and self.items[i] < weight: # find position the element goes
        i += 1

    self.items.append(0) # extends the array
    for j in range(len(self.items) - 1, i, -1):
        # shifts the 0 to where the weight is to be inserted then inserts the weight
        self.items[j] = self.items[j-1]
    self.items[i] = weight
```

Figure 11: Sorted array data structure

Q2: Implement an efficient method to retrieve an item based on its weight.

An efficient method to retrieve an item based off its weight was done using binary search. This can be seen below:

```
def find_item(self, weight) :  
    if not self.items:  
        return None  
  
    # binary search  
    low = 0  
    high = len(self.items) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if self.items[mid] == weight:  
            return weight  
        elif self.items[mid] < weight:  
            low = mid + 1  
        else:  
            high = mid - 1  
  
    # Return closest found element  
    if low == 0:  
        return self.items[0]  
    if low >= len(self.items):  
        return self.items[-1]  
  
    before = self.items[low - 1] # before final iteration  
    after = self.items[low] # after final iteration  
  
    if abs(before - weight) <= abs(after - weight):  
        return before  
    else:  
        return after
```

Figure 12: Binary search to find an item based on its weight

Q3: If an exact match is not found, return the closest available item.

If the exact element was not found, it would return the closest available item. This was done by comparing the value before and after where this item would be. Shown below:

```
if abs(before - weight) <= abs(after - weight):  
    return before  
else:  
    return after
```

Figure 13: Returning closest available item

The full implementation of this warehouse searching system can be found in the appendix section of this report.

Task 2: Handling warehouse organisation overtime

Q1: After a few hundred items have been delivered, we observe a slowdown in the searching process. Investigate the cause of this slowdown.

A slowdown was noticed when a large number of items were added. Especially in the removing items and adding items process. This could be because a linear search is done to find the item that is needed to be removed, and is also used to find the position where the new element is to be added. A linear search runs in $O(n)$ time. Seen below is the linear search for elements

```
def remove_item(self, weight):
    for i in range(len(self.items)):
        if self.items[i] == weight: # find item to remove
            for j in range(i, len(self.items) - 1): # shifts item to end of array
                self.items[j] = self.items[j+1]
            self.items.pop() # removes the end element of the array
            return
    print("item not found")
```

Figure 14: Linear search to find item to be removed

```
def add_item(self, weight):
    i = 0
    while i < len(self.items) and self.items[i] < weight:
        i += 1
```

Figure 15: Linear search to find index of where to add an item

Q2: Propose and implement an improved version of the original solution to maintain efficiency over time.

This can be improved by changing the data structure to a self-balancing binary search tree. This will guarantee all operations remain efficient over time. This tree maintains its height at $O(\log n)$, via rotations after every insert or delete. Adding an item becomes $O(\log n)$, removing an item is $O(\log n)$, and finding an item is $O(\log n)$. In the sorted array approach, add and remove required $O(n)$ element shifts in the worst case. This AVL method ensures $O(\log n)$ time complexity for all operations at all times. This ensures even at extremely high amounts of Warehouse items, these operations remain efficient. Below is my implementation of the improvement.

```

class AVLNode:
    """
    Node in a self balancing binary tree
    Attributes:
    key: the value stored in the node
    left/ right: left/ right child of subtree
    height: height of this subtree
    """
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1 # leaf node

# return node height
def height(node):
    return node.height if node else 0

# recompute node height from children
def updateHeight(node):
    node.height = 2 + max(height(node.left), height(node.right))

# balance factor = height of left - height of right
def balanceFactor(node):
    return height(node.left) - height(node.right)

# Perform right rotation around node y
def rotateRight(y):
    x = y.left
    T2 = x.right
    x.right = y
    y.left = T2
    updateHeight(y)
    updateHeight(x)
    return x

# Perform left rotation around node x
def rotateLeft(x):
    y = x.right
    T2 = y.left
    y.left = x
    x.right = T2
    updateHeight(x)
    updateHeight(y)
    return y

# Rebalances subtree if balance factor is > 1 or < -1
def rebalance(node):
    updateHeight(node)
    bf = balanceFactor(node)

    # left heavy
    if bf > 1:
        # left-right case
        if balanceFactor(node.left) < 0:
            node.left = rotateLeft(node.left)
        # left-left case
        return rotateRight(node)

    # right-heavy
    if bf < -1:
        # right-left case
        if balanceFactor(node.right) > 0:
            node.right = rotateRight(node.right)
        return rotateLeft(node)

    return node # already balanced

class AVLTree:
    # Self balancing BST

    def __init__(self):
        self.root = None

    def insert(self, key):
        def _insert(node, key):
            if node is None:
                return AVLNode(key)
            if key < node.key:
                node.left = _insert(node.left, key)
            else:
                node.right = _insert(node.right, key)
            return rebalance(node)
        self.root = _insert(self.root, key)

    def delete(self, key):

```

```

# removes item from AVL tree if present
def minNode(node):
    while node.left:
        node = node.left
    return node

def _delete(node, key):
    if node is None:
        return None
    if key < node.key:
        node.left = _delete(node.left, key)
    elif key > node.key:
        node.right = _delete(node.right, key)
    else:
        # Node to delete found
        if node.left is None:
            return node.right
        if node.right is None:
            return node.left

        # two children: replace with in-order successor
        successor = minNode(node.right)
        node.key = successor.key
        node.right = _delete(node.right, successor.key)
    return rebalance(node) if node else None
self.root = _delete(self.root, key)

# Finds item with exact weight
def search(self, key):
    node = self.root
    while node:
        if key == node.key:
            return node
        node = node.left if key < node.key else node.right
    return None

def findClosest(self, key):
    # returns item closest to weight
    node = self.root
    floor = None
    ceil = None
    while node:
        if key == node.key:
            return node.key
        if key < node.key:
            ceil = node.key
            node = node.left
        else:
            floor = node.key
            node = node.right
    if floor is None:
        return ceil
    if ceil is None:
        return floor
    return floor if (key - floor) <= (ceil - key) else ceil

class Warehouse:
    def __init__(self):
        self.tree = AVLTree()

    # adds an item to the wearhouse
    def addItem(self, weight):
        self.tree.insert(weight)

    # removes an item from the wearhouse
    def removeItem(self, weight):
        if not self.tree.search(weight):
            print("Error: weight not found.")
        self.tree.delete(weight)

    # finds an item from the warehouse
    def findItem(self, weight):
        if self.tree.root is None:
            return None
        node = self.tree.search(weight)
        return node.key if node else self.tree.findClosest(weight)

# Example usage.
wh = Warehouse()
for w in [5, 2, 8, 1, 6]:
    wh.addItem(w)

print(wh.findItem(4))

```

```
(base) jackmillington@Jacks-MacBook-Pro A  
rithims/Assignment/5  
(base) jackmillington@Jacks-MacBook-Pro A
```

Benchmarking:

I benchmarked all 3 operations on the time it took to complete 10000 operations on a Wearhouse with 10000 items. Here are the results:

```
(base) jackmillington@Jacks-MacBook-Pro A  
rithims/Assignment/problem 3/problem3.py'  
Performed 10000 operations each:  
  add_item:    5.610140 seconds total  
  find_item:   0.012143 seconds total  
  remove_item: 2.723704 seconds total  
(base) jackmillington@Jacks-MacBook-Pro A
```

Figure 16: Results on original wearhouse

```
Performed 10000 operations each:  
  add_item:    0.045080 seconds total  
  find_item:   0.008990 seconds total  
  remove_item: 0.052780 seconds total  
(base) jackmillington@Jacks-MacBook-Pro A
```

Figure 17: Results on improved wearhouse version

As can be seen through the results, the new improved implementation significantly outperformed the original Wearhouse implementation across every operation. This proves the theory that it would be faster, clearly showing it maintains efficiency over time.

###-----END PROBLEM 3-----###

V. Problem 4

A navigation system is designed to help users find the best route between two locations while considering both distance and toll costs. The system represents roads as edges (with distances) and intersections as nodes (which may have toll fees). Some intersections have no toll fees (toll = 0), while others require payment.

Your task is to implement a system that calculates multiple route options for a user, allowing them to choose between:

- The shortest route (minimizing distance only).
- The most cost-effective route (minimizing toll fees).
- A balanced route that considers both distance and toll fees.

Additionally, routes that are both long and expensive should be automatically discarded.

Task 1: Implement a Route Selection System:

Q1: Model the road network as a graph where edges represent roads (with distances as weights). Nodes represent intersections (with associated toll costs).

The road network was modelled as a graph. The edges represent the roads with the distances as weights. The nodes represent the intersections, with the associated toll costs. Therefore, this was correctly implemented.

```
# road network modelled as a graph
roadNetwork = {
  "A": {"toll": 0, "roads": {"B": 4, "C": 8}},
  "B": {"toll": 2, "roads": {"A": 4, "C": 2, "D": 5}},
  "C": {"toll": 3, "roads": {"A": 8, "B": 2, "D": 3, "E": 6}},
  "D": {"toll": 2, "roads": {"B": 5, "C": 3, "E": 2}},
  "E": {"toll": 5, "roads": {"C": 6, "D": 2}}
}
```

Figure 18: Road network modelled as graph

Q2: Implement Dijkstra's Algorithm to find the shortest path.

Dijkstra's algorithm was then implemented to find the shortest path, distance wise only. This can be seen below:

```

1  def dijkstrasAlgo(graph, start, end):
2
3      # initialises all distances to infinity except the start
4      distances = {}
5      for node in graph:
6          distances[node] = float('inf')
7      distances[start] = 0
8
9      # initialises previous dictionary
10     # {"A": None, "B": None, etc}
11     previous = {}
12     for node in graph:
13         previous[node] = None
14
15     # creates a set {"A", "B", etc}
16     unvisited = set(graph.keys())
17
18     while unvisited: # while there are still unvisited nodes
19         current = None
20         for node in unvisited: # loops through all unvisited and finds node with least distance
21             if current is None or distances[node] < distances[current]:
22                 current = node
23
24         if current == end: # if node is the end node, stop
25             break
26
27         # visits all neighbours of current node
28         for neighbor, weight in graph[current]["roads"].items():
29             if neighbor in unvisited:
30                 # find dist from start to this neighbour
31                 newDistance = distances[current] + weight
32                 # if it is the shortest found so far, update record in distances
33                 # and record we came from current
34                 if newDistance < distances[neighbor]:
35                     distances[neighbor] = newDistance
36                     previous[neighbor] = current
37
38         unvisited.remove(current) # mark as visited and repeat for all nodes
39
40     # reconstruct path
41     path = []
42     current = end # start at end node
43     while current is not None:
44         path.insert(0, current) # inserts to front of path list to keep in forward order
45         current = previous[current]
46
47     # if the start isnt reached then there is no path
48     if distances[end] == float('inf'):
49         return None, []
50
51     return distances[end], path # returns a distance and the path it took
52

```

Figure 19: Dijkstras Algorithm on distance only

Q3: Modify the algorithm to account for toll fees and generate multiple route

options

Due to the addition of toll fees, Dijkstra's algorithm had to account for both distance and toll fees, therefore, multiple route options were generated. One route ignored the tolls completely, only finding the shortest distance. Another ignored the distance, only finding the route with the least number of tolls. The third route added both together and found the route with the lowest sum of the distance and tolls combined, when both weighted equally. The fourth found the route with the lowest sum of the two, when the distance was weighted double. Finally the fifth option found the route with the lowest sum of the two when the sum was weighted double.

Below is how I implemented this change:

First I modified the function parameters to include an option to change the weightings of distance and toll, and changed all 'distances' to a generic 'weights'

```
def dijkstrasAlgo(graph, start, end, weightType = 'distance', distWeight = 1.0, tollWeight = 1.0):
```

Figure 20: Implemented weightType, distWeight, and tollWeight

This logic here filters the calculation based on the weight type:

```
# visits all neighbours of current node
for neighbor, distance in graph[current]["roads"].items():
    if neighbor in unvisited:
        toll = graph[neighbor]["toll"]
        if weightType == 'distance':
            # find distance from start to this neighbour
            newWeight = weights[current] + distance
        if weightType == 'toll':
            # find toll from start to this neighbour
            newWeight = weights[current] + toll
        if weightType == 'balance':
            # find distance and toll from start to this neighbour weighted accordingly
            newWeight = weights[current] + (distance*distWeight + toll*tollWeight)
```

Figure 21: Weight type logic

This function calculates the distance and toll amounts based on a given path in a graph.

```
def calcDistAndToll(graph, path):
    dist = 0
    toll = 0
    for i in range(len(path)):
        node = path[i]
        toll += graph[node]['toll']
        if i > 0:
            prev = path[i - 1]
            dist += graph[prev]['roads'][node]
    return dist, toll, path
```

Figure 22: calcDistAndToll function

Finally, this function prints the multiple options out for the user to see.

```
def printOptions(graph, start, stop):
    options = []
    dist, toll, path = calcDistAndToll(graph, dijkstrasAlgo(graph, start, stop, 'distance'))
    options.append((dist, toll, path))
    dist, toll, path = calcDistAndToll(graph, dijkstrasAlgo(graph, start, stop, 'toll'))
    options.append((dist, toll, path))
    dist, toll, path = calcDistAndToll(graph, dijkstrasAlgo(graph, start, stop, 'balance', 1, 1))
    options.append((dist, toll, path))
    dist, toll, path = calcDistAndToll(graph, dijkstrasAlgo(graph, start, stop, 'balance', 2, 1))
    options.append((dist, toll, path))
    dist, toll, path = calcDistAndToll(graph, dijkstrasAlgo(graph, start, stop, 'balance', 1, 2))
    options.append((dist, toll, path))

    i = 0
    for dist, toll, path in options:
        i+=1
        print(f"Option {i}: distance: {dist}, toll: {toll}, Path is {path}")
    return
```

Figure 23: printOptions function

Here is the output, showing the user 5 different options.

```
● (base) jackmillington@Jacks-MacBook-Pro Assignment % /usr/local/
ear 2 Tri 1/Computing Algorithms/Assignment/problem 4/problem4t
Option 1: distance: 11, toll: 9, Path is ['A', 'B', 'D', 'E']
Option 2: distance: 14, toll: 8, Path is ['A', 'C', 'E']
Option 3: distance: 11, toll: 9, Path is ['A', 'B', 'D', 'E']
Option 4: distance: 11, toll: 9, Path is ['A', 'B', 'D', 'E']
Option 5: distance: 11, toll: 9, Path is ['A', 'B', 'D', 'E']
○ (base) jackmillington@Jacks-MacBook-Pro Assignment %
```

Figure 24: Task 1 output

Task 2: Filtering Inefficient routes:

Q1: Discard any routes that are significantly longer and more expensive than alternative routes.

Any routes that were significantly longer and more expensive than alternative were discarded. This was defined as a route that had both a longer distance and a total toll that was more than another route. Additionally, if any were a duplicate of each other, one would be discarded as there is no reason to show the user two of the exact same route. This filter implementation can be seen below.

```
def filterOptions(options):
    # first, filter for duplicates
    seen = set()
    filtered = []
    for dist, toll, path in options:
        # set as tuple so it can be put in set
        pathTuple = tuple(path)
        if pathTuple not in seen:
            filtered.append((dist, toll, path))
            seen.add(pathTuple)

    # Then delete any paths that are both longer and cost more than another
    good = []
    for i, (dist_i, toll_i, path_i) in enumerate(filtered):
        worse = False
        for j, (dist_j, toll_j, path_j) in enumerate(filtered):
            if i == j:
                continue
            if (dist_i >= dist_j and toll_i >= toll_j) and (dist_i > dist_j or toll_i > toll_j):
                worse = True
                break
        if not worse:
            good.append((dist_i, toll_i, path_i))

    return good
```

Figure 25: Duplicate and worse path filtering

Q2: Allow users to compare remaining routes and choose their preferred option.

This leaves the user with up to 5 distinct routes to choose from, and make their decision based on how much they valued travel distance over toll costs. Below is the output based on the example given in the spec.

```
Option 1: distance: 11, toll: 9, Path is ['A', 'B', 'D', 'E']
Option 2: distance: 14, toll: 8, Path is ['A', 'C', 'E']
```

Figure 26: Problem 4 output

###-----END PROBLEM 4-----###

Appendix

1. Problem 1

```
# Task 1
class ArrayQueue:
    # Initialises queue as an array
    def __init__(self):
        self.queue = []
    # adds order to end of queue
    def addOrder(self, order):
        self.queue.append(order)
    # pops the element at the front of the queue
    def processOrder(self):
        if self.isEmpty():
            return None
        return self.queue.pop(0)
    # views the element at the front of the queue
    def viewNextOrder(self):
        return self.queue[0]
    # removes a specified order from the queue,
    # moves everything behind it forwards by 1 in the queue
    def cancelOrder(self, order):
        if order in self.queue:
            self.queue.remove(order)
            return True
        return False
    # checks if the queue is empty
    def isEmpty(self):
        return len(self.queue) == 0
```

```
# Task 4:
class Node:
    # initialises a node with an order
    # and an empty pointer to the next
    def __init__(self, order):
        self.order = order
        self.next = None

class SinglyLinkedListQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    # adds an order to the back of the queue
    def addOrder(self, order):
        new_node = Node(order)
        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node
    # returns order at front of queue and
    # sets the front to the next node in queue
    def processOrder(self):
        if self.isEmpty():
            return None
        temp = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        return temp.order
    # returns the order at the front
    def viewNextOrder(self):
        if self.isEmpty():
            return None
        return self.front.order
    # linear search to find order,
    # then removes it from queue,
    # and sets pointer of node in front to node
    # behind the one that just got cancelled
    def cancelOrder(self, order):
        if self.isEmpty():
            return False
        temp = self.front
        prev = None
        while temp is not None:
            if temp.order == order:
                if prev is None:
                    self.front = temp.next
                else:
                    prev.next = temp.next
                if temp == self.rear:
                    self.rear = prev
                return True
            prev = temp
            temp = temp.next
```

```

        prev = temp
        temp = temp.next
    return False
# Checks if queue is empty
def isEmpty(self):
    return self.front is None

class DllNode:
    """
    Node in a doubly linked list.
    Attributes:
        order: The value stored in this node.
        prev: Reference to the previous node.
        next: Reference to the next node.
    """
    def __init__(self, order):
        self.order = order
        self.prev = None
        self.next = None

class DoublyLinkedListQueue:
    """
    Queue implemented via a doubly linked list.
    Operations:
        - addOrder(order)      : O(1), returns a direct node reference
        - processOrder()       : O(1), removes and returns front order
        - viewNextOrder()      : O(1), peeks at the front order
        - cancelOrder(node)    : O(1), removes given node reference
        - isEmpty()            : O(1), checks empty
    """
    def __init__(self):
        self.front = None
        self.rear = None

    def addOrder(self, order):
        """Add `order` to the back of the queue; return node reference."""
        new_node = DllNode(order)
        if not self.front: # empty queue
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            new_node.prev = self.rear
            self.rear = new_node
        return new_node

    def processOrder(self):
        """Remove and return the front order, or None if empty."""
        if self.isEmpty():
            return None
        node = self.front
        self.front = node.next
        if self.front:
            self.front.prev = None
        else:
            self.rear = None # queue is now empty
        return node.order

    def viewNextOrder(self):
        """Return the front order without removing it, or None if empty."""
        return self.front.order if not self.isEmpty() else None

    def cancelOrder(self, node):
        """
        Remove the node referenced by `node` in O(1) time.
        Assumes `node` is currently in this queue.
        """
        # Re-link prev and next around node
        if node.prev:
            node.prev.next = node.next
        else:
            self.front = node.next

        if node.next:
            node.next.prev = node.prev
        else:
            self.rear = node.prev

        # Clear node's pointers (optional cleanup)
        node.prev = node.next = None
        return True

    def isEmpty(self):
        """Return True if queue is empty."""
        return self.front is None

```

Benchmarking:

```
import time
import random

def benchmark_queues(N):
    data = list(range(N))
    random.shuffle(data)

    # Process Order benchmark
    arrq = ArrayQueue()
    for x in data:
        arrq.addOrder(x)
    start = time.perf_counter()
    for _ in range(N):
        arrq.processOrder()
    time_arr_process = time.perf_counter() - start

    llq = SinglyLinkedListQueue()
    for x in data:
        llq.addOrder(x)
    start = time.perf_counter()
    for _ in range(N):
        llq.processOrder()
    time_ll_process = time.perf_counter() - start

    dllq = DoublyLinkedListQueue()
    for x in data:
        dllq.addOrder(x)
    start = time.perf_counter()
    for _ in range(N):
        dllq.processOrder()
    time_dll_process = time.perf_counter() - start

    # View Next Order benchmark
    arrq = ArrayQueue()
    for x in data:
        arrq.addOrder(x)
    start = time.perf_counter()
    for _ in range(N):
        arrq.viewNextOrder()
    time_arr_view = time.perf_counter() - start

    llq = SinglyLinkedListQueue()
    for x in data:
        llq.addOrder(x)
    start = time.perf_counter()
    for _ in range(N):
        llq.viewNextOrder()
    time_ll_view = time.perf_counter() - start

    dllq = DoublyLinkedListQueue()
    for x in data:
        dllq.addOrder(x)
    start = time.perf_counter()
    for _ in range(N):
        dllq.viewNextOrder()
    time_dll_view = time.perf_counter() - start

    # Cancel Order benchmark
    arrq = ArrayQueue()
    for x in data:
        arrq.addOrder(x)
    start = time.perf_counter()
    for x in data:
        arrq.cancelOrder(x)
    time_arr_cancel = time.perf_counter() - start

    llq = SinglyLinkedListQueue()
    for x in data:
        llq.addOrder(x)
    start = time.perf_counter()
    for x in data:
        llq.cancelOrder(x)
    time_ll_cancel = time.perf_counter() - start

    dllq = DoublyLinkedListQueue()
    nodes = [dllq.addOrder(x) for x in data]
    start = time.perf_counter()
    for node in nodes:
        dllq.cancelOrder(node)
    time_dll_cancel = time.perf_counter() - start

    # isEmpty benchmark
    arrq = ArrayQueue()
    for x in data:
        arrq.addOrder(x)
    start = time.perf_counter()
    for _ in range(N):
        arrq.isEmpty()
```

```

time_arr_empty = time.perf_counter() - start

llq = SinglyLinkedListQueue()
for x in data:
    llq.addOrder(x)
start = time.perf_counter()
for _ in range(N):
    llq.isEmpty()
time_ll_empty = time.perf_counter() - start

dllq = DoublyLinkedListQueue()
for x in data:
    dllq.addOrder(x)
start = time.perf_counter()
for _ in range(N):
    dllq.isEmpty()
time_dll_empty = time.perf_counter() - start

# Print results
print(f"{'Operation':<20}{'Array':>12}{'SinglyLL':>12}{'DoublyLL':>12}")
print("-" * 56)
print(f"{'processOrder':<20}{time_arr_process:12.8f}{time_ll_process:12.8f}{time_dll_process:12.8f}")
print(f"{'viewNextOrder':<20}{time_arr_view:12.8f}{time_ll_view:12.8f}{time_dll_view:12.8f}")
print(f"{'cancelOrder':<20}{time_arr_cancel:12.8f}{time_ll_cancel:12.8f}{time_dll_cancel:12.8f}")
print(f"{'isEmpty':<20}{time_arr_empty:12.8f}{time_ll_empty:12.8f}{time_dll_empty:12.8f}")

if __name__ == "__main__":
    benchmark_queues(10000)

```

Operation	Array	SinglyLL	DoublyLL
processOrder	0.00596508	0.00089004	0.00094396
viewNextOrder	0.00025879	0.00055429	0.00051175
cancelOrder	0.00491821	0.00099329	0.00056696
isEmpty	0.00038771	0.00023363	0.00022883

2. Problem 2

```

# Task 1:
import random
import time
import matplotlib.pyplot as plt

# Quick Sort:
def quickSort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[random.randint(0, len(arr) - 1)]
    l = []
    e = []
    g = []
    for num in arr:
        if num > pivot:
            g.append(num)
        elif num < pivot:
            l.append(num)
        else:
            e.append(num)

    leftSorted = quickSort(l)
    rightSorted = quickSort(g)
    return leftSorted + e + rightSorted

# MergeSort
def mergesort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = mergesort(arr[:mid])
    right = mergesort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result += left[i:]
    result += right[j:]
    return result

```

```

        else:
            result.append(right[j])
            j += 1
        result.extend(left[i:])
        result.extend(right[j:])
        return result

# Heapsort
def heapsort(arr):
    def heapify(arr, n, i):
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and arr[i] < arr[left]:
            largest = left
        if right < n and arr[largest] < arr[right]:
            largest = right
        if largest != i:
            arr[i], arr[largest] = arr[largest], arr[i]
            heapify(arr, n, largest)

    n = len(arr)
    for i in range(n // 2, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
    return arr

# Insertion Sort
def insertionsort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

# Selection Sort
def selectionsort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

#####
# BENCHMARKING

# Generate test data
def generate_data(size, data_type):
    if data_type == 'random':
        return [random.randint(0, 10000) for _ in range(size)]
    elif data_type == 'reverse':
        return [i for i in range(size, 0, -1)]
    elif data_type == 'outliers':
        return [random.randint(0, 100) for _ in range(size - 10)] + [10000] * 10
    elif data_type == 'duplicates':
        return [random.randint(0, 100) for _ in range(size)]

# Benchmarking sorting algorithms
def benchmark_sorting_algorithms():
    sizes_large = [1000, 5000, 10000, 20000, 30000, 40000, 50000, 100000, 500000]
    sizes_small = [1000, 5000, 10000, 20000] # Added 20000 as the extra size for testing insertion and selection sort
    data_types = ['random', 'reverse', 'outliers', 'duplicates']
    results = {data_type: [] for data_type in data_types}

    for data_type in data_types:
        for size in sizes_large:
            data = generate_data(size, data_type)
            start_time = time.time()
            quickSort(data.copy())
            quicksort_time = time.time() - start_time

            start_time = time.time()
            mergesort(data.copy())
            mergesort_time = time.time() - start_time

            start_time = time.time()
            heapsort(data.copy())
            heapsort_time = time.time() - start_time

            result = {'size': size, 'quick': quicksort_time, 'merge': mergesort_time, 'heap': heapsort_time}

```

```

        if size in sizes_small:
            start_time = time.time()
            insertionsort(data.copy())
            result['insertion'] = time.time() - start_time

            start_time = time.time()
            selectionsort(data.copy())
            result['selection'] = time.time() - start_time

        results[data_type].append(result)

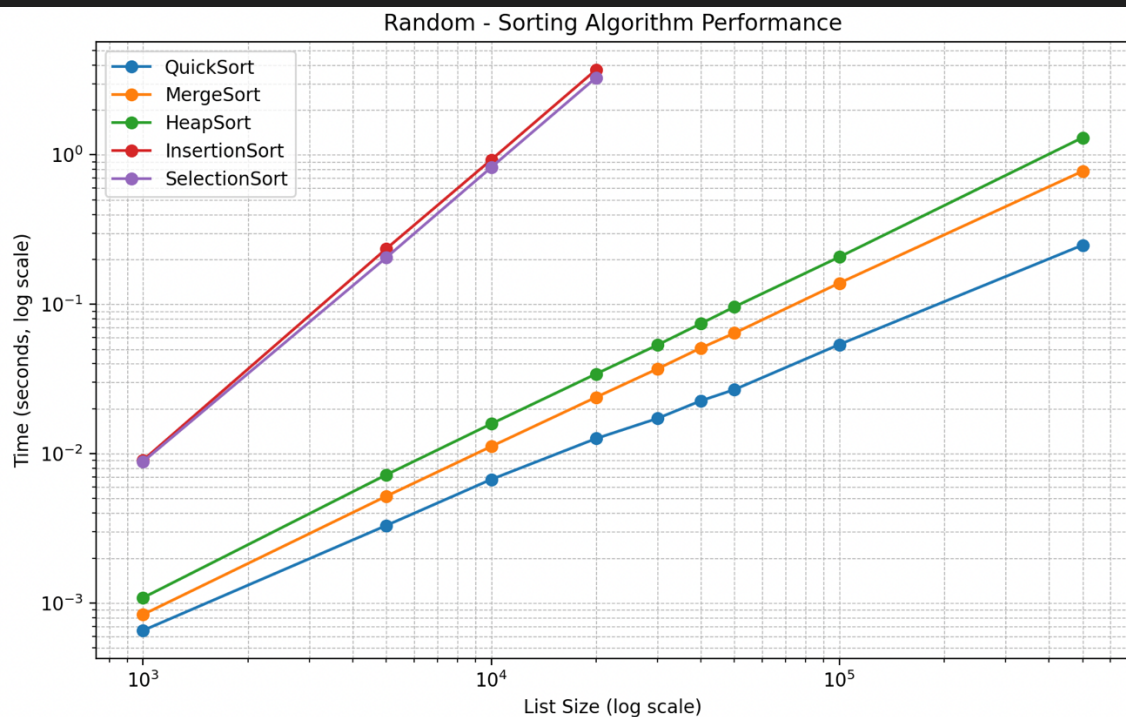
    return results

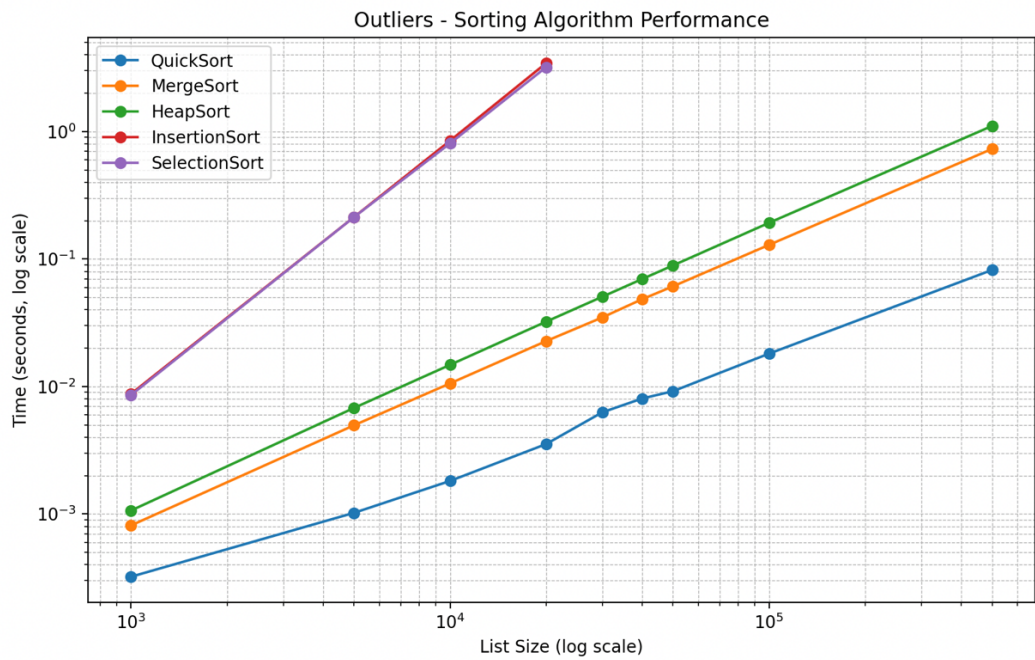
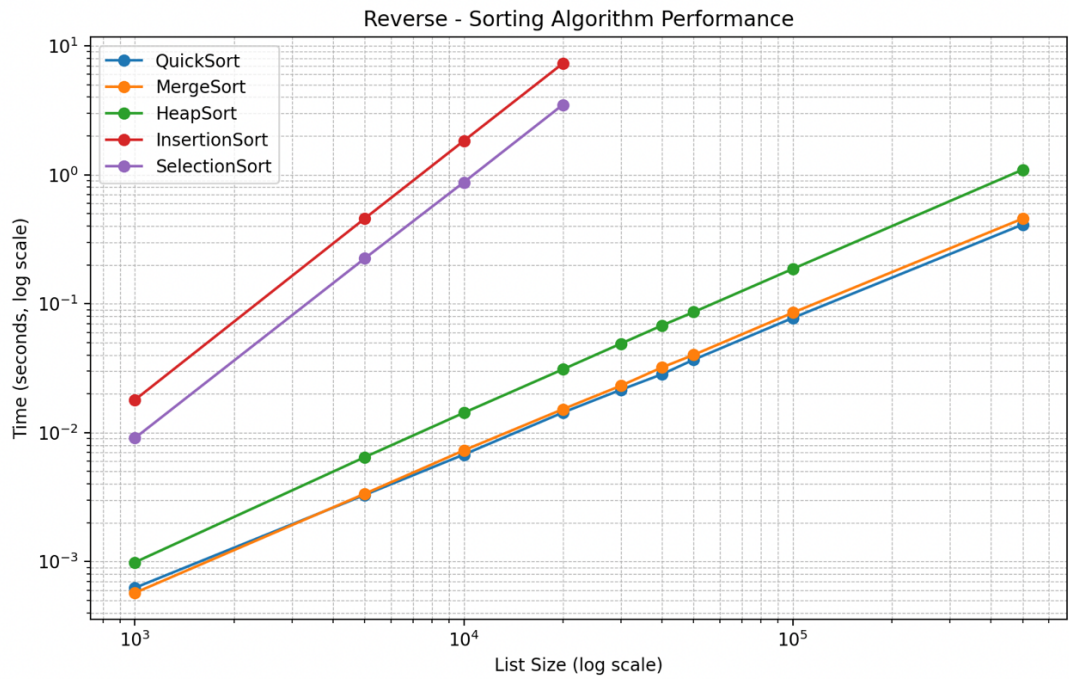
# Visualize results with log scale for x and y axes
def plot_results(results):
    for data_type in results.keys():
        sizes = [result['size'] for result in results[data_type]]
        plt.figure(figsize=(10, 6))
        plt.plot(sizes, [result['quick'] for result in results[data_type]], label='QuickSort', marker='o')
        plt.plot(sizes, [result['merge'] for result in results[data_type]], label='MergeSort', marker='o')
        plt.plot(sizes, [result['heap'] for result in results[data_type]], label='HeapSort', marker='o')
        if 'insertion' in results[data_type][0]:
            plt.plot(sizes[:4], [result['insertion'] for result in results[data_type][:4]], label='InsertionSort',
marker='o')
        plt.plot(sizes[:4], [result['selection'] for result in results[data_type][:4]], label='SelectionSort',
marker='o')

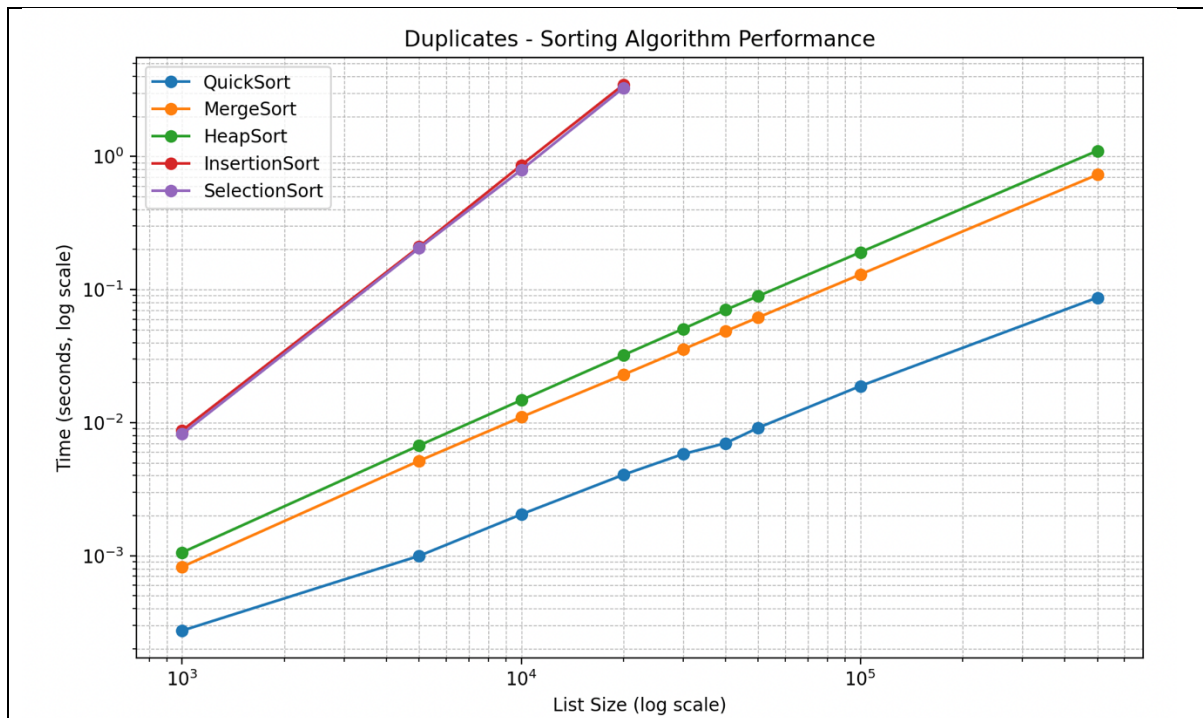
        plt.xscale('log')
        plt.yscale('log')
        plt.xlabel('List Size (log scale)')
        plt.ylabel('Time (seconds, log scale)')
        plt.title(f'{data_type.capitalize()} - Sorting Algorithm Performance')
        plt.legend()
        plt.grid(True, which='both', linestyle='--', linewidth=0.5)
        plt.show()

# Run benchmark test and plot the results
results = benchmark_sorting_algorithms()
plot_results(results)

```







Improved QuickSort:

```
import random
import time
import matplotlib.pyplot as plt

# QuickSort Algorithm
def quickSort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[random.randint(0, len(arr) - 1)]
    l, e, g = [], [], []
    for num in arr:
        if num > pivot:
            g.append(num)
        elif num < pivot:
            l.append(num)
        else:
            e.append(num)
    return quickSort(l) + e + quickSort(g)

# Insertion Sort for small subarrays
def insertion_sort(arr, left, right):
    for i in range(left + 1, right + 1):
        key = arr[i]
        j = i - 1
        while j >= left and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

def median_of_three(arr, left, right):
    indices = random.sample(range(left, right + 1), 3)
    values = [arr[i] for i in indices]
    values.sort()
    return values[1]

def quick_sort_helper(arr, left, right):
    if right - left < 10:
        insertion_sort(arr, left, right)
        return

    pivot = median_of_three(arr, left, right)
    l, e, g = [], [], []
    for num in arr[left:right + 1]:
        if num > pivot:
            g.append(num)
        elif num < pivot:
            l.append(num)
        else:
            e.append(num)
    arr[left:right + 1] = quickSortn(l) + e + quickSortn(g)

def quickSortn(arr):
    quick_sort_helper(arr, 0, len(arr) - 1)
    return arr
```

```

# Generate test data
def generate_data(size, data_type):
    if data_type == 'random':
        return [random.randint(0, 10000) for _ in range(size)]
    elif data_type == 'reverse':
        return [i for i in range(size, 0, -1)]
    elif data_type == 'outliers':
        return [random.randint(0, 100) for _ in range(size - 10)] + [10000] * 10
    elif data_type == 'duplicates':
        return [random.randint(0, 100) for _ in range(size)]

# Benchmarking sorting algorithms
def benchmark_sorting_algorithms():
    sizes_larger = [1000, 5000, 10000, 20000, 30000, 40000, 50000, 100000, 500000, 1000000, 2000000]
    sizes_small = [1000, 5000, 10000, 20000]
    data_types = ['random', 'reverse', 'outliers', 'duplicates']
    results = {data_type: [] for data_type in data_types}

    for data_type in data_types:
        for size in sizes_larger:
            data = generate_data(size, data_type)

            start_time = time.time()
            quickSort(data.copy())
            quicksort_time = time.time() - start_time

            start_time = time.time()
            quickSortn(data.copy())
            quicksortn_time = time.time() - start_time

            result = {'size': size, 'quick': quicksort_time, 'quickn': quicksortn_time}

            if size in sizes_small:
                start_time = time.time()
                insertion_sort(data.copy(), 0, len(data) - 1)
                result['insertion'] = time.time() - start_time

            results[data_type].append(result)

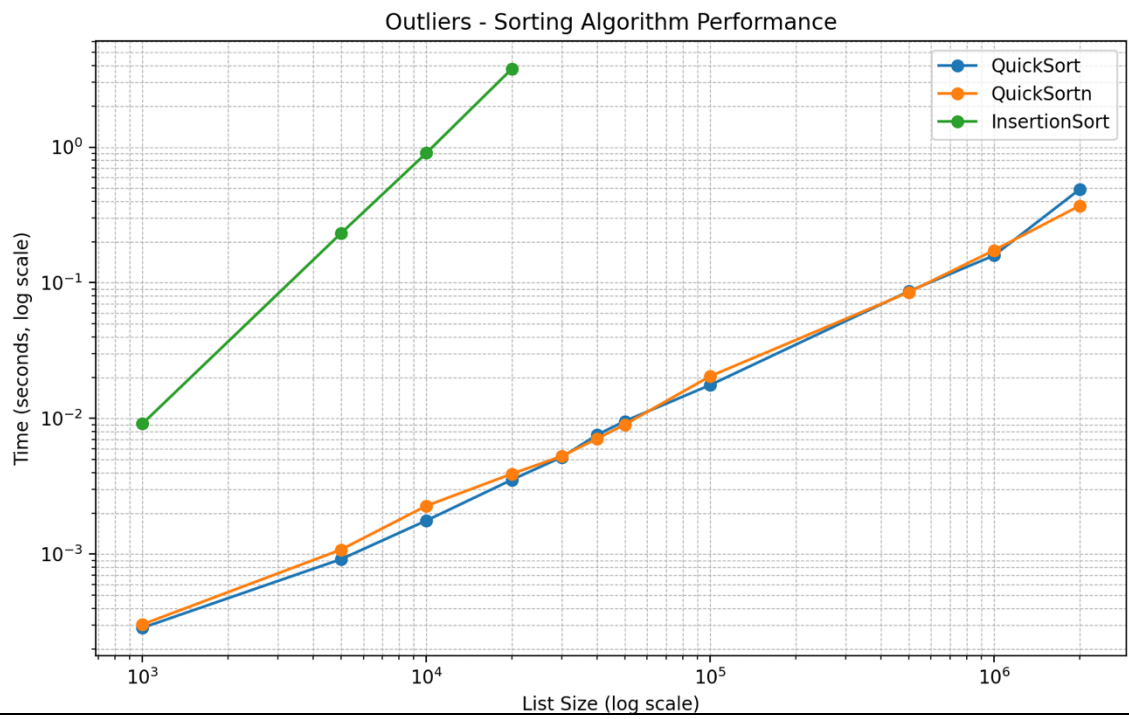
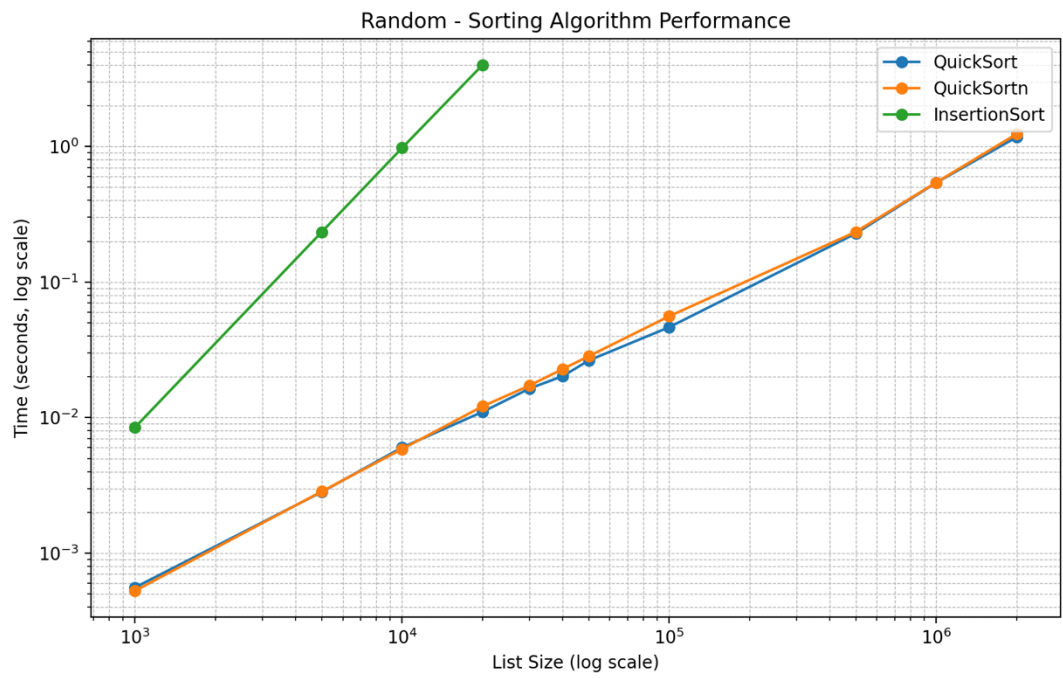
    return results

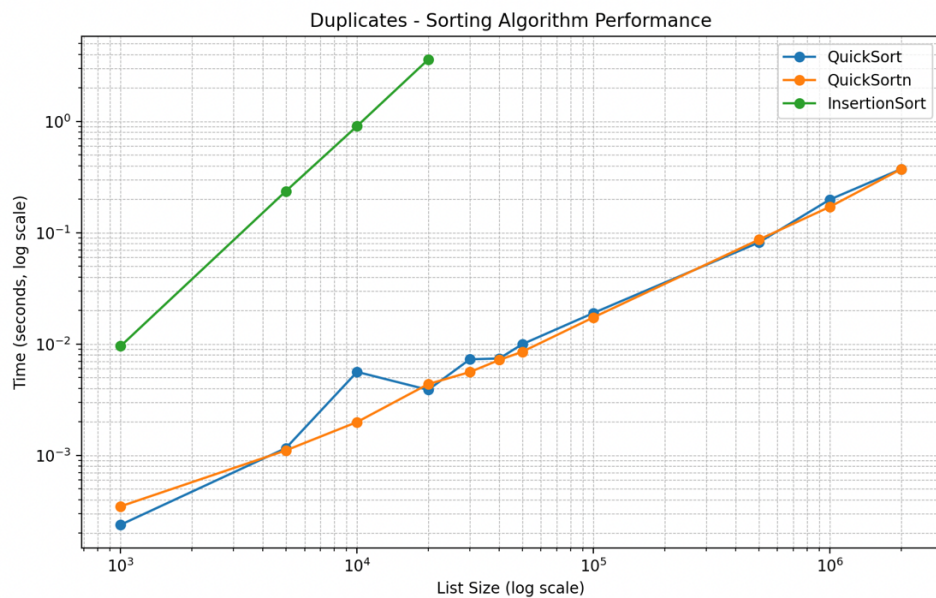
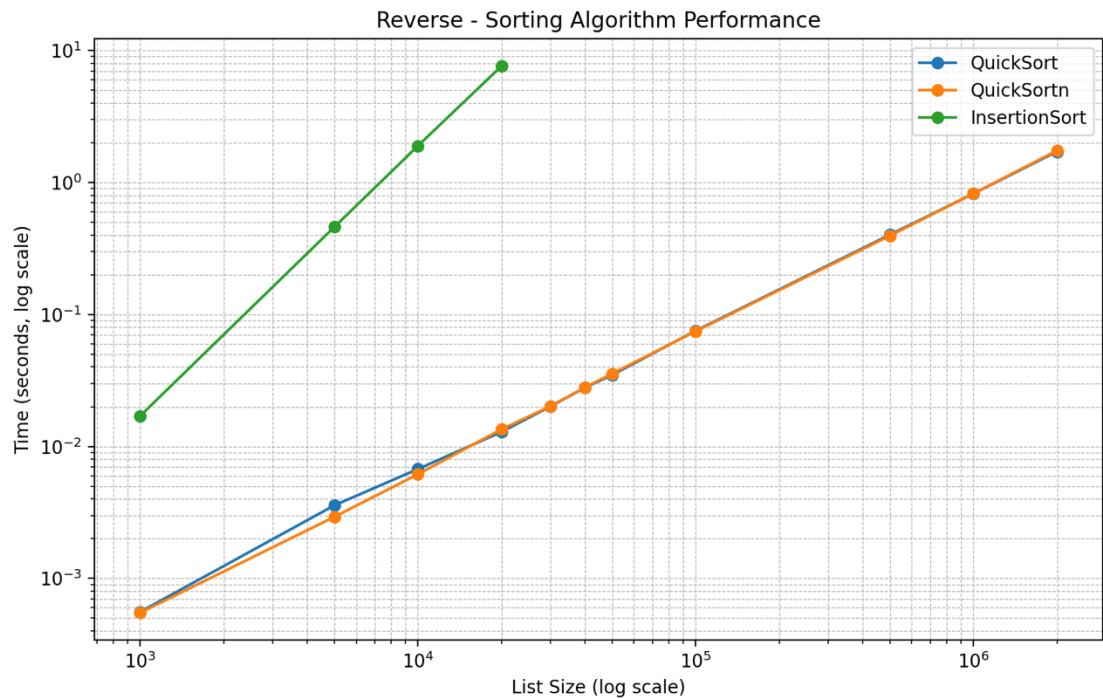
# Visualize results
def plot_results(results):
    for data_type in results.keys():
        sizes = [result['size'] for result in results[data_type]]
        plt.figure(figsize=(10, 6))
        plt.plot(sizes, [result['quick'] for result in results[data_type]], label='QuickSort', marker='o')
        plt.plot(sizes, [result['quickn'] for result in results[data_type]], label='QuickSortn', marker='o')
        if 'insertion' in results[data_type][0]:
            plt.plot(sizes[:4], [result['insertion'] for result in results[data_type][:4]], label='InsertionSort',
                    marker='o')

        plt.xscale('log')
        plt.yscale('log')
        plt.xlabel('List Size (log scale)')
        plt.ylabel('Time (seconds, log scale)')
        plt.title(f'{data_type.capitalize()} - Sorting Algorithm Performance')
        plt.legend()
        plt.grid(True, which='both', linestyle='--', linewidth=0.5)
        plt.show()

# Run benchmark and plot results
results = benchmark_sorting_algorithms()
plot_results(results).

```





3. Problem 3

```
class Warehouse:
    def __init__(self):
        self.items = []

    def add_item(self, weight):
        i = 0
        while i < len(self.items) and self.items[i] < weight: # find position the element goes
            i += 1

        self.items.append(0) # extends the array
        for j in range(len(self.items) - 1, i, -1):
```

```

        # shifts the 0 to where the weight is to be inserted then inserts the weight
        self.items[j] = self.items[j-1]
        self.items[i] = weight

    def remove_item(self, weight):
        for i in range(len(self.items)):
            if self.items[i] == weight: # find item to remove
                for j in range(i, len(self.items) - 1): # shifts item to end of array
                    self.items[j] = self.items[j+1]
                self.items.pop() # removes the end element of the array
                return
        print("item not found")

    def find_item(self, weight) :
        if not self.items:
            return None

        # binary search
        low = 0
        high = len(self.items) - 1
        while low <= high:
            mid = (low + high) // 2
            if self.items[mid] == weight:
                return weight
            elif self.items[mid] < weight:
                low = mid + 1
            else:
                high = mid - 1

        # Return closest found element
        if low == 0:
            return self.items[0]
        if low >= len(self.items):
            return self.items[-1]

        before = self.items[low - 1] # before final iteration
        after = self.items[low] # after final iteration

        if abs(before - weight) <= abs(after - weight):
            return before
        else:
            return after

    def display_items(self):
        print(self.items)

```

Improvement:

```

class AVLNode:
    """
    Node in a self balancing binary tree
    Attributes:
    key: the value stored in the node
    left/ right: left/ right child of subtree
    height: height of this subtree
    """
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1 # leaf node

# return node height
def height(node):
    return node.height if node else 0

# recompute node height from children
def updateHeight(node):
    node.height = 2 + max(height(node.left), height(node.right))

# balance factor = height of left - height of right
def balanceFactor(node):
    return height(node.left) - height(node.right)

# Perform right rotation around node y
def rotateRight(y):
    x = y.left
    T2 = x.right
    x.right = y
    y.left = T2
    updateHeight(y)
    updateHeight(x)
    return x

# Perform left rotation around node x
def rotateLeft(x):
    y = x.right
    T2 = y.left
    y.left = x
    x.right = T2

```

```

        updateHeight(x)
        updateHeight(y)
        return y

# Rebalances subtree if balance factor is > 1 or < -1
def rebalance(node):
    updateHeight(node)
    bf = balanceFactor(node)

    # left heavy
    if bf > 1:
        # left-right case
        if balanceFactor(node.left) < 0:
            node.left = rotateLeft(node.left)
        # left-left case
        return rotateRight(node)

    # right-heavy
    if bf < -1:
        # right-left case
        if balanceFactor(node.right) > 0:
            node.right = rotateRight(node.right)
        return rotateLeft(node)

    return node # already balanced

class AVLTree:
    # Self balancing BST

    def __init__(self):
        self.root = None

    def insert(self, key):
        def _insert(node, key):
            if node is None:
                return AVLNode(key)
            if key < node.key:
                node.left = _insert(node.left, key)
            else:
                node.right = _insert(node.right, key)
            return rebalance(node)
        self.root = _insert(self.root, key)

    def delete(self, key):
        # removes item from AVL tree if present
        def minNode(node):
            while node.left:
                node = node.left
            return node

        def _delete(node, key):
            if node is None:
                return None
            if key < node.key:
                node.left = _delete(node.left, key)
            elif key > node.key:
                node.right = _delete(node.right, key)
            else:
                # Node to delete found
                if node.left is None:
                    return node.right
                if node.right is None:
                    return node.left

                # two children: replace with in-order successor
                successor = minNode(node.right)
                node.key = successor.key
                node.right = _delete(node.right, successor.key)
            return rebalance(node) if node else None
        self.root = _delete(self.root, key)

    # Finds item with exact weight
    def search(self, key):
        node = self.root
        while node:
            if key == node.key:
                return node
            node = node.left if key < node.key else node.right
        return None

    def findClosest(self, key):
        # returns item closest to weight
        node = self.root
        floor = None
        ceil = None
        while node:
            if key == node.key:
                return node.key
            if key < node.key:

```

```

        ceil = node.key
        node = node.left
    else:
        floor = node.key
        node = node.right
    if floor is None:
        return ceil
    if ceil is None:
        return floor
    return floor if (key - floor) <= (ceil - key) else ceil

class Warehouse:
    def __init__(self):
        self.tree = AVLTree()

    # adds an item to the warehouse
    def addItem(self, weight):
        self.tree.insert(weight)

    # removes an item from the warehouse
    def removeItem(self, weight):
        if not self.tree.search(weight):
            print("Error: weight not found.")
        self.tree.delete(weight)

    # finds an item from the warehouse
    def findItem(self, weight):
        if self.tree.root is None:
            return None
        node = self.tree.search(weight)
        return node.key if node else self.tree.findClosest(weight)

# Example usage.
wh = Warehouse()
for w in [5, 2, 8, 1, 6]:
    wh.addItem(w)

print(wh.findItem(4))

```

```

● (base) jackmillington@Jacks-MacBook-Pro Assignment % /usr/local/bin/python3 "/Users/jackmillington/Documents/Uni/Year 2 Tri 1/Computing Algorithms/Assignment/problem 3/improvement2.py"
5

```

Benchmarking:

```

import time
import random

def benchmark_operations(N):
    # Generate N unique elements and shuffle
    data = list(range(N))
    random.shuffle(data)

    # Preload the warehouse with N items
    wh = Warehouse()
    for x in data:
        wh.addItem(x)

    # 1) Benchmark add_item N times (adding new keys N..2N-1)
    add_keys = [x + N for x in data]
    start = time.perf_counter()
    for key in add_keys:
        wh.addItem(key)
    total_add = time.perf_counter() - start

    # 2) Benchmark find_item N times on random keys in [0..2N-1]
    start = time.perf_counter()
    for _ in range(N):
        key = random.randint(0, 2*N - 1)
        wh.findItem(key)
    total_find = time.perf_counter() - start

    # 3) Benchmark remove_item N times (removing the keys we just added)
    start = time.perf_counter()
    for key in add_keys:
        wh.removeItem(key)
    total_remove = time.perf_counter() - start

    print(f"Performed {N} operations each:")
    print(f"  add_item:      {total_add:.6f} seconds total")
    print(f"  find_item:     {total_find:.6f} seconds total")
    print(f"  remove_item:   {total_remove:.6f} seconds total")

if __name__ == "__main__":
    N = 10_000
    benchmark_operations(N)

```

Performed 10000 operations each:

add_item: 0.045080 seconds total

find_item: 0.008990 seconds total

remove_item: 0.052780 seconds total

rithims/Assignment/problem 3/problem3.py

Performed 10000 operations each:

add_item: 5.610140 seconds total

find_item: 0.012143 seconds total

remove_item: 2.723704 seconds total

4. Problem 4

```
def dijkstrasAlgo(graph, start, end, weightType = 'distance', distWeight = 1.0, tollWeight = 1.0):

    # initialises all weights to infinity except the start
    weights = {}
    for node in graph:
        weights[node] = float('inf')
    weights[start] = 0

    # initialises previous dictionary
    # {"A": None, "B": None, etc}
    previous = {}
    for node in graph:
        previous[node] = None

    # creates a set {"A", "B", etc}
    unvisited = set(graph.keys())

    while unvisited: # while there are still unvisited nodes
        current = None
        for node in unvisited: # loops through all unvisited and finds node with least weight
            if current is None or weights[node] < weights[current]:
                current = node

        if current == end: # if node is the end node, stop
            break

        # visits all neighbours of current node
        for neighbor, distance in graph[current]["roads"].items():
            if neighbor in unvisited:
                toll = graph[neighbor]["toll"]
                if weightType == 'distance':
                    # find distance from start to this neighbour
                    newWeight = weights[current] + distance
                if weightType == 'toll':
                    # find toll from start to this neighbour
                    newWeight = weights[current] + toll
                if weightType == 'balance':
                    # find distance and toll from start to this neighbour weighted accordingly
                    newWeight = weights[current] + (distance*distWeight + toll*tollWeight)

                # if it is the shortest found so far, update record in weights
                # and record we came from current
                if newWeight < weights[neighbor]:
                    weights[neighbor] = newWeight
                    previous[neighbor] = current

        unvisited.remove(current) # mark as visited and repeat for all nodes

    # reconstruct path
    path = []
    current = end # start at end node
    while current is not None:
        path.insert(0, current) # inserts to front of path list to keep in forward order
        current = previous[current]

    # if the start isnt reached then there is no path
    if weights[end] == float('inf'):
        return None, []
```

```

    return path # returns a weight and the path it took

def calcDistAndToll(graph, path):
    dist = 0
    toll = 0
    for i in range(len(path)):
        node = path[i]
        toll += graph[node]['toll']
        if i > 0:
            prev = path[i - 1]
            dist += graph[prev]['roads'][node]
    return dist, toll, path

def printOptions(graph, start, stop):
    options = []
    dist, toll, path = calcDistAndToll(graph, dijkstrasAlgo(graph, start, stop, 'distance'))
    options.append((dist, toll, path))
    dist, toll, path = calcDistAndToll(graph, dijkstrasAlgo(graph, start, stop, 'toll'))
    options.append((dist, toll, path))
    dist, toll, path = calcDistAndToll(graph, dijkstrasAlgo(graph, start, stop, 'balance', 1, 1))
    options.append((dist, toll, path))
    dist, toll, path = calcDistAndToll(graph, dijkstrasAlgo(graph, start, stop, 'balance', 2, 1))
    options.append((dist, toll, path))
    dist, toll, path = calcDistAndToll(graph, dijkstrasAlgo(graph, start, stop, 'balance', 1, 2))
    options.append((dist, toll, path))

    result = filterOptions(options)
    i = 0
    for dist, toll, path in result:
        i+=1
        print(f"Option {i}: distance: {dist}, toll: {toll}, Path is {path}")
    return

def filterOptions(options):
    # first, filter for duplicates
    seen = set()
    filtered = []
    for dist, toll, path in options:
        # set as tuple so it can be put in set
        pathTuple = tuple(path)
        if pathTuple not in seen:
            filtered.append((dist, toll, path))
            seen.add(pathTuple)

    # Then delete any paths that are both longer and cost more than another
    good = []
    for i, (dist_i, toll_i, path_i) in enumerate(filtered):
        worse = False
        for j, (dist_j, toll_j, path_j) in enumerate(filtered):
            if i == j:
                continue
            if (dist_i >= dist_j and toll_i >= toll_j) and (dist_i > dist_j or toll_i > toll_j):
                worse = True
                break
        if not worse:
            good.append((dist_i, toll_i, path_i))

    return good

# road network modelled as a graph
roadNetwork = {
    "A": {"toll": 0, "roads": {"B": 4, "C": 8}},
    "B": {"toll": 2, "roads": {"A": 4, "C": 2, "D": 5}},
    "C": {"toll": 3, "roads": {"A": 8, "B": 2, "D": 3, "E": 6}},
    "D": {"toll": 2, "roads": {"B": 5, "C": 3, "E": 2}},
    "E": {"toll": 5, "roads": {"C": 6, "D": 2}}
}

# example usage
printOptions(roadNetwork, "A", "E")

```

(base) jackmillington@jacks-mbp Assignment % /usr/local/bin/python3.11.2 --help
 ngton/Documents/Uni/Year 2 Tri 1/Computing Algorithms/Assignment 1
 rue.py"
 Option 1: distance: 11, toll: 9, Path is ['A', 'B', 'D', 'E']
 Option 2: distance: 14, toll: 8, Path is ['A', 'C', 'E']
 (base) jackmillington@jacks-mbp Assignment %